ANKARA ÜNİVERSİTESİ
MÜHENDİSLİK FAKÜLTESİ
BİLGİSAYAR MÜHENDİSLİĞİ BÖLÜMÜ

BLM4522 AĞ TABANLI PARALEL DAĞITIM SİSTEMLERİ
VİZE RAPORU

Elif POLAT
22290192

Nisan, 2026

Github:
https://github.com/elf321/BLM4522-Projects

İÇİNDEKİLER

Proje 7 - Veritabanı Yedekleme ve Otomasyon Çalışması 2
Veri Seti: DVD Rental Database 2
Kullanılan Teknolojiler 2
Hazırlık Aşaması 2
Aşama 1: Manuel Yedekleme 2
Aşama 2: Scripting ve Parametrik Yapı 3
Aşama 3: Cron ile Otomatik Yedekleme 3
Aşama 4: Gelişmiş Otomatik Yedekleme ve Bildirim 3
Sonuç 4
Proje 2 - Veritabanı Yedekleme ve Felaketten Kurtarma Planı 5
Kullanılan Ek Teknolojiler 5
Hazırlık Aşaması ve WAL Konfigürasyonu 5
Aşama 1: Tam Yedekleme 5
Aşama 2: WAL Konfigürasyonu 5
Aşama 3: Otomatik Yedekleme ve Cron 6
Aşama 4: Point-in-Time Recovery 6
Aşama 5: Felaket Kurtarma Testi 6
Sonuç ve Genel Değerlendirme **7**

Proje 7 - Veritabanı Yedekleme ve Otomasyon Çalışması

Bu çalışma, **PostgreSQL** veritabanı üzerinde manuel yedekleme süreçlerinden başlayarak, bu süreçlerin **Bash Scripting** ile otomatize edilmesini, Cron servisleri ile zamanlanmasını ve yöneticiye anlık bildirim gönderen modern bir altyapının kurulmasını kapsamaktadır.

Veri Seti: DVD Rental Database

PostgreSQL'in standart eğitim veri seti olan dvdrental, bir DVD kiralama mağazasının iş süreçlerini modeller. Veritabanı; 15 tablo, çok sayıda tetikleyici , saklı yordam ve karmaşık ilişkisel bağlar içerir. Gerçek dünya senaryolarını (müşteri yönetimi, envanter takibi, ödeme geçmişi) simüle ettiği için yedekleme stratejilerinin test edilmesi adına kritik bir öneme sahiptir.

Kullanılan Teknolojiler

PostgreSQL 15: Yüksek performanslı, açık kaynaklı ilişkisel veritabanı yönetim sistemi.
Bash (Shell Scripting): İşletim sistemiyle doğrudan konuşabilen, düşük kaynak tüketen otomasyon dili.
Cron: Unix sistemlerin standart zamanlayıcıs.
AppleScript (osascript): Sistem seviyesinde UI bildirimleri tetiklemek için kullanılan köprü.
pg_dump: Veritabanını tutarlı bir şekilde paketleyen endüstri standardı araç.

Hazırlık Aşaması

Projenin ilk adımı, PostgreSQL ekosisteminde standart bir kompleks yapı örneği olan **dvdrental** veri setinin sisteme entegre edilmesiyle başlamıştır. Ardından, veritabanının tablo yapısını, anahtar ilişkilerini ve veri yoğunluğunu analiz etmek amacıyla DataGrip kullanılmıştır. DataGrip üzerindeki diyagramlar aracılığıyla tablolar arası Foreign Key bağları incelenmiş ve yedekleme stratejisinin ne denli kritik olduğu teknik olarak doğrulanmıştır. Bu ön analiz süreci, veritabanının sağlıklı çalıştığının SQL sorgularıyla teyit edilmesiyle tamamlanmış ve otomasyon aşamasına geçiş için güvenli bir zemin oluşturulmuştur.

Aşama 1 : Manuel Yedekleme

İlk aşamada bağlantı parametreleri test edilmiştir. pg_dump aracılığıyla terminalde çalıştırılan komut sonrasında backups klasörüne dump uzantılı dosya başarılı bir şekilde kaydedilmiştir.

Aşama 2: Scripting ve Parametrik Yapı

Manuel süreçlerin ardından, yedekleme işlemini standartlaştırmak için **Bash Script** (.sh) mimarisine geçilmiştir. Script, mkdir -p komutu sayesinde gerekli backups ve logs dizinlerinin varlığını her çalışma öncesinde denetlemekte, eğer dizinler mevcut değilse otomatik olarak oluşturarak dosya sistemini hazırlamaktadır. Öncelikle, yedek ve log dosyalarının isimlerinde kullanılmak üzere güncel tarih ve saat bilgisi oluşturulur. Ardından veritabanı adı, kullanıcı, bağlantı bilgileri ve dosyaların kaydedileceği dizinler tanımlanır. Yedeklerin ve logların tutulacağı klasörler yoksa oluşturulur. Daha sonra pg_dump aracı kullanılarak belirtilen veritabanının yedeği alınır ve hem çıktılar hem de olası hata mesajları bir log dosyasına yazılır. İşlem tamamlandıktan sonra komutun başarılı olup olmadığı kontrol edilir; başarılıysa yedek dosyasının yolu ekrana yazdırılır, başarısız olduysa log dosyasının incelenmesi gerektiğini belirten bir hata mesajı gösterilir. Bu sayede yedekleme süreci düzenli, takip edilebilir ve otomatik bir şekilde yönetilmiş olur.

Aşama 3: Cron ile Otomatik Yedekleme

Bu aşamada, hazırlanan yedekleme scriptinin otomatik olarak çalıştırılması amacıyla terminal üzerinden **crontab -e** komutu kullanılarak bir cron görevi tanımlanmıştır. Normal şartlarda bu işlemin daha uzun zaman aralıklarında gerçekleştirilmesi tercih edilse de, zaman kısıtı nedeniyle test edilebilmesi için scriptin her dakika çalışacak şekilde ayarlanması sağlanmıştır. Crontab dosyasına eklenen bu yapı sayesinde sistem, belirlenen zaman aralığında yedekleme işlemini düzenli olarak yürütmüştür. Script her çalıştığında oluşturulan yedek dosyalarının *backups* dizinine kaydedildiği, işlem detaylarının ise *logs* klasöründeki log dosyasına yazıldığı gözlemlenmiştir. Bu sayede yedekleme süreci otomatikleştirilmiş; sürekli, kesintisiz ve izlenebilir bir yapıya kavuşturulmuştur.

Aşama 4: Gelişmiş Otomatik Yedekleme ve Bildirim

Bu aşamada, yedekleme işlemi yine crontab üzerinden zamanlanarak otomatik şekilde çalıştırılmıştır. Script çalıştığında veritabanının yedeği alınmış, elde edilen dosya belirlenen *backups* dizinine kaydedilmiş ve işlem sürecine ait tüm detaylar *logs* klasöründeki log dosyasına yazılmıştır. Önceki aşamadan farklı olarak bu adımda, yedekleme işleminin sonucu daha ayrıntılı biçimde raporlanmış; başarılı olması durumunda yedek dosyasının adı ve boyutu gibi bilgiler loglara eklenmiştir. Ayrıca sistem, işlem sonucuna göre kullanıcıya bildirim gönderecek şekilde yapılandırılmıştır. Yedekleme başarılı olduğunda bilgilendirici bir bildirim gösterilirken, hata durumunda uyarı mesajı ve sesli bildirim ile kullanıcı haberdar edilmiştir. Böylece yedekleme süreci yalnızca otomatik hale getirilmekle kalmamış, aynı zamanda izlenebilirliği ve kullanıcıya geri bildirim sağlama kapasitesi de artırılmıştır.

Sonuç

Bu çalışma kapsamında, PostgreSQL üzerinde bulunan dvdrental veritabanı için güvenilir bir yedekleme altyapısı tasarlanmış ve aşamalı olarak geliştirilmiştir. Öncelikle manuel olarak alınan yedekler üzerinden sistemin doğruluğu test edilmiş, ardından Bash Script ile süreç parametrik ve otomatik çalışabilir hale getirilmiştir. Cron servisinin devreye alınmasıyla birlikte yedekleme işlemi belirli aralıklarla insan müdahalesi olmadan gerçekleştirilecek şekilde zamanlanmış; son aşamada ise sistem, başarılı veya başarısız sonuçları gerçek zamanlı bildirimlerle yöneticilere iletebilen gelişmiş bir yapıya kavuşturulmuştur. Böylece yedekleme süreci hem süreklilik hem de izlenebilirlik açısından güçlendirilmiş; bir veritabanı yönetim sisteminde olması gereken temel güvenlik ve sürdürülebilirlik kriterleri başarılı bir şekilde yerine getirilmiştir.

Proje 2 - Veritabanı Yedekleme ve Felaketten Kurtarma Planı

Bu çalışma, PostgreSQL veritabanı üzerinde veri kaybını sıfıra indirmeyi hedefleyen **Point-in-Time Recovery (PITR)** mekanizmasının kurulmasını, **WAL (Write-Ahead Logging)** arşivleme stratejilerini ve olası felaket senaryolarına karşı veritabanı geri yükleme testlerini kapsamaktadır.

Kullanılan Ek Teknolojiler

WAL (Write-Ahead Logging): Her işlemin kalıcı hafızaya yazılmadan önce kaydedildiği günlükleme sistemi.

Hazırlık Aşaması ve WAL Konfigürasyonu

Projenin temel taşı, veritabanının "kara kutusu" olarak adlandırılan WAL arşivleme sisteminin aktif edilmesidir. postgresql.conf dosyası üzerinde yapılan düzenlemelerle archive_mode aktif edilmiş ve her bir log segmentinin güvenli bir wal_archive dizinine kopyalanması sağlanmıştır. Bu sayede, iki tam yedek arasında gerçekleşen tüm işlemlerin izi sürülerek "sıfır veri kaybı" hedeflenmiştir.

Aşama 1: Tam Yedekleme

Projenin başlangıç aşamasında, veritabanının mevcut tüm yapısını ve verilerini içeren mantıksal bir tam yedekleme işlemi gerçekleştirilmiştir. pg_dump aracı kullanılarak yürütülen bu süreçte, veritabanının belirli bir andaki "fotoğrafı" çekilerek .dump formatında bir arşiv dosyası oluşturulmuştur. Bu aşama, sistemin en temel güvenlik katmanını oluşturmakla birlikte, daha sonra yapılacak olan zaman tabanlı kurtarma (PITR) işlemleri için vazgeçilmez bir referans noktası işlevi görmektedir. Manuel olarak alınan bu yedeklerin tutarlılığı, dosya boyutu ve içerik kontrolleri yapılarak doğrulanmış ve bir sonraki aşama olan sürekli arşivleme için güvenli bir zemin hazırlanmıştır.

Aşama 2: WAL Konfigürasyonu

Sadece periyodik olarak alınan tam yedeklerin, iki yedekleme süresi arasında gerçekleşen işlemleri koruyamadığı saptanmış, bu riskin bertaraf edilmesi için WAL (Write-Ahead Logging) konfigürasyonu devreye alınmıştır. PostgreSQL konfigürasyon dosyaları üzerinde yapılan düzenlemelerle archive_mode aktif edilmiş ve her bir log segmentinin güvenli bir arşiv dizinine kopyalanması sağlanmıştır. Bu sayede veritabanında gerçekleşen her bir veri girişi, güncelleme veya silme işlemi anlık olarak kayıt altına alınmaya başlanmıştır. Bu yapılandırma, veritabanının sadece son yedek anına değil, bu yedekten sonraki herhangi bir saniyeye de geri döndürülebilmesine olanak tanıyan teknik altyapıyı sağlamıştır.

Aşama 3: Otomatik Yedekleme ve Cron

Yedekleme süreçlerinin sürdürülebilirliğini sağlamak ve insan faktöründen kaynaklanabilecek aksaklıkları gidermek amacıyla sistem otomatize edilmiştir. Bu amaçla hazırlanan Bash scriptleri, veritabanı bağlantı bilgilerini, dosya isimlendirme standartlarını ve hata ayıklama mekanizmalarını içerecek şekilde parametrik bir yapıda kurgulanmıştır. Hazırlanan bu otomasyon aracı, Unix sistemlerin standart zamanlayıcısı olan Cron servisine bağlanarak periyodik bir çalışma takvimine oturtulmuştur. Her çalışma sonrasında başarı durumuna göre log dosyalarının oluşturulması ve yedeklerin tarih damgalarıyla arşivlenmesi sayesinde, sistem yöneticisinin müdahalesine gerek kalmadan kendi kendini yöneten bir veri koruma döngüsü oluşturulmuştur.

Aşama 4: Point-in-Time Recovery (PITR - Zaman Tabanlı Kurtarma)

Projenin en kritik aşamalarından biri olan bu adımda, veritabanının geçmişteki belirli bir ana geri döndürülme kapasitesi test edilmiştir. Bu senaryoda, tam yedek alındıktan sonra veritabanına yeni veriler eklenmiş ve ardından bilinçli bir "felaket" yaratılarak tablolar silinmiştir. Yapılan testlerde, sadece tam yedeği geri yüklemenin son eklenen verileri kurtarmada yetersiz kaldığı somut olarak gözlemlenmiştir. Ancak tam yedek dosyası ile Aşama 2'de biriktirilen WAL logları birleştirilerek, veritabanı tam olarak tablonun silindiği saniyeden hemen öncesine döndürülmüştür. Bu süreç, kurumların veri kaybı toleransını sıfıra indiren bir teknoloji olarak başarıyla simüle edilmiştir.

Aşama 5: Felaket Kurtarma Testi

Son aşamada, kurulan sistemin farklı senaryolara karşı dayanıklılığı test edilerek Disaster Recovery planı doğrulanmıştır. Bu kapsamda sadece tüm tablonun silinmesi değil, aynı zamanda yanlışlıkla silinen tek bir veri satırının ana sisteme zarar vermeden kurtarılması senaryosu üzerinde çalışılmıştır. Kurtarma denemeleri sırasında karşılaşılan Foreign Key kısıtlamaları ve fonksiyon eksiklikleri gibi teknik engeller, veritabanı bütünlüğünü bozmadan aşılmış ve sistemin esnekliği kanıtlanmıştır. Yapılan tüm testler, kurulan altyapının hem büyük ölçekli sistem çökmelerinde hem de küçük ölçekli kullanıcı hatalarında veri tutarlılığını saniyeler içinde geri kazandırabildiğini ortaya koymuştur.

Sonuç ve Genel Değerlendirme

Bu çalışma, modern bir veritabanı yönetiminde yedekleme ve kurtarmanın bir bütün olarak ele alınması gerektiğini göstermiştir. Uygulanan 5 aşamalı strateji ile veritabanı; fiziksel hatalara, yazılımsal sorunlara ve insan kaynaklı yanlış işlemlere karşı tam korumalı hale getirilmiştir. GitHub üzerinde paylaşılan scriptler ve konfigürasyonlar, projenin teknik uygulanabilirliğini ve otomasyon başarısını belgelemektedir.